

```

"""Tests for uma.semantic - Engine3-correct architecture."""

import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))

import numpy as np
import pytest

from uma.core.state import FieldPosterior
from uma.semantic.friction import SemanticFriction, SemanticFrictionConfig
from uma.semantic.constraints import (
    LinearConstraint, BallConstraint, QuadraticConstraint, ConstraintSet
)
from uma.semantic.inarticulation import NullInarticulator, ProjectionInarticulator
from uma.semantic.ir import UMA_IR, IRNode
from uma.semantic.engine import SemanticEngine, SemanticEngineConfig, tokenize_to

# — Fixtures —————

@pytest.fixture
def rng(): return np.random.default_rng(0)

@pytest.fixture
def dim(): return 16

@pytest.fixture
def z_target(dim, rng): return rng.standard_normal(dim)

@pytest.fixture
def full_posterior(dim, rng):
    mu = rng.standard_normal(dim)
    A = rng.standard_normal((dim, dim))
    return FieldPosterior.full(mu, A @ A.T + np.eye(dim) * 0.1)

# — Friction: decimal walk —————

class TestFriction:
    def test_walks_down_by_decimals(self, full_posterior, z_target):
        sf = SemanticFriction(z_target, SemanticFrictionConfig(step_weight=0.1))
        sf.reset(full_posterior)
        prev = 1.0
        for i in range(10):

```

```

        rec = sf.update(full_posterior, t=i * 0.04)
        assert rec.friction <= prev + 1e-9
        prev = rec.friction

def test_friction_starts_at_one(self, full_posterior, z_target):
    sf = SemanticFriction(z_target)
    sf.reset(full_posterior)
    assert sf.value == 1.0

def test_friction_never_goes_negative(self, full_posterior, z_target):
    sf = SemanticFriction(z_target, SemanticFrictionConfig(step_weight=0.5))
    sf.reset(full_posterior)
    for i in range(20):
        rec = sf.update(full_posterior, t=i * 0.04)
    assert sf.value >= 0.0

def test_summary_keys(self, full_posterior, z_target):
    sf = SemanticFriction(z_target)
    sf.reset(full_posterior)
    sf.update(full_posterior, t=0.0)
    s = sf.summary()
    assert "friction_final" in s
    assert "dz_dt_final" in s
    assert "sf_total" in s

# — Constraints —————

class TestConstraints:
    def test_linear_satisfied(self, dim, rng):
        a = rng.standard_normal(dim); a /= np.linalg.norm(a)
        c = LinearConstraint(a, 1.5)
        z = c.project(rng.standard_normal(dim))
        assert abs(c.check(z)) < 1e-10

    def test_ball_projects_to_surface(self, dim, rng):
        c = BallConstraint(r=1.0, center=np.zeros(dim))
        z_proj = c.project(5.0 * np.ones(dim))
        assert abs(np.linalg.norm(z_proj) - 1.0) < 1e-10

    def test_quadratic_feasible_unchanged(self, dim, rng):
        c = QuadraticConstraint(kappa=1000.0, Q=None, center=np.zeros(dim))
        z = rng.standard_normal(dim)
        if c.check(z) < 1e-6:
            np.testing.assert_allclose(c.project(z), z, atol=1e-12)

    def test_default_set_has_three_named_constraints(self, dim, z_target):

```

```
cs = ConstraintSet.default_semantic_constraints(dim, z_target)
names = [c.name for c in cs.constraints]
assert "alpha_dIse" in names and "beta_VH" in names and "gamma_Jd4m" in n
```

```
# — IR —————
```

```
class TestIR:
    def test_append_and_len(self):
        ir = UMA_IR()
        ir.append(IRNode("n1", "evolve", {"dt": 0.04}))
        ir.append(IRNode("n2", "checkpoint", {}))
        assert len(ir) == 2

    def test_summary_string(self):
        ir = UMA_IR()
        ir.append(IRNode("n1", "evolve", {}))
        ir.append(IRNode("n2", "observe", {}))
        ir.append(IRNode("n3", "checkpoint", {}))
        s = ir.summary()
        assert "evolve" in s and "checkpoint" in s
```

```
# — Engine3 bridge —————
```

```
class TestEngine3Bridge:
    def test_binary_weight_is_positive(self):
        x = tokenize_to_binary_weight("hello")
        assert x > 0

    def test_anchor_gives_one(self):
        x = tokenize_to_binary_weight("dIse", anchor="dIse")
        assert abs(x - 1.0) < 1e-9

    def test_string_to_observation_shape(self):
        obs, y = string_to_observation("test string", dim=8)
        assert y.shape == (8,)
        assert y[0] > 0
```

```
# — Full engine integration —————
```

```
class TestSemanticEngine:
    def _setup(self):
        from uma import Config, UMAClient
        cfg = Config()
        rng = np.random.default_rng(1)
```

```

N = cfg.grid.N
client = UMAClient(cfg)
psi0 = 0.2 * (rng.standard_normal((N, N)) + 1j * rng.standard_normal((N,
client.initialize(psi0, sigma0=0.15)
z_target = np.zeros(client.projection.real_dim)
return client, z_target

def test_runs_without_error(self):
    client, z_target = self._setup()
    engine = SemanticEngine(z_target)
    result = engine.run(client, n_steps=10)
    assert result is not None

def test_ir_compiled(self):
    client, z_target = self._setup()
    engine = SemanticEngine(z_target)
    result = engine.run(client, n_steps=10)
    assert len(result.ir) > 0

def test_friction_walks_down(self):
    client, z_target = self._setup()
    engine = SemanticEngine(z_target, SemanticEngineConfig(
        friction=SemanticFrictionConfig(step_weight=0.1))
    result = engine.run(client, n_steps=20)
    frictions = [r.friction for r in result.friction_records]
    assert frictions[-1] < frictions[0] or frictions[-1] == 0.0

def test_uninitialized_raises(self):
    from uma import Config, UMAClient
    client = UMAClient(Config())
    engine = SemanticEngine(np.zeros(4))
    with pytest.raises(RuntimeError):
        engine.run(client, n_steps=5)

def test_summary_string(self):
    client, z_target = self._setup()
    engine = SemanticEngine(z_target)
    result = engine.run(client, n_steps=10)
    s = result.summary()
    assert "SemanticEngine" in s and "Friction" in s

def test_production_metrics_sane(self):
    client, z_target = self._setup()
    engine = SemanticEngine(z_target)
    result = engine.run(client, n_steps=15)
    pm = result.production_metrics
    assert 0.0 <= pm.semantic_density <= 1.0

```

```

    assert pm.trajectory_delta >= 0.0
    assert pm.stage in ("ingestion", "stability", "inarticulation", "solve")

def test_engine3_text_input(self):
    client, z_target = self._setup()
    dim = len(z_target)
    obs, y = string_to_observation("THRUPUT field solve", dim=dim)
    engine = SemanticEngine(z_target)
    result = engine.run(client, n_steps=15, observations=[(obs, y)])
    assert result.friction_records # got checkpoints

if __name__ == "__main__":
    print("Running semantic tests...")
    tf = TestFriction()
    rng_ = np.random.default_rng(0)
    mu_ = rng_.standard_normal(16)
    A_ = rng_.standard_normal((16, 16))
    p_ = FieldPosterior.full(mu_, A_ @ A_.T + np.eye(16) * 0.1)
    z_ = rng_.standard_normal(16)
    tf.test_walks_down_by_decimals(p_, z_);    print(" friction decimal walk: PA
    tf.test_friction_never_goes_negative(p_, z_); print(" friction floor: PASS")

    tc = TestConstraints()
    tc.test_linear_satisfied(16, rng_);    print(" linear constraint: PASS")
    tc.test_ball_projects_to_surface(16, rng_); print(" ball constraint: PASS")

    tb = TestEngine3Bridge()
    tb.test_binary_weight_is_positive();    print(" binary weight: PASS")
    tb.test_string_to_observation_shape(); print(" string→obs bridge: PASS")

    te = TestSemanticEngine()
    te.test_runs_without_error(); print(" engine runs: PASS")
    te.test_ir_compiled();          print(" IR compiled: PASS")
    te.test_friction_walks_down(); print(" friction walks: PASS")
    te.test_engine3_text_input(); print(" Engine3 text input: PASS")
    print("\nAll tests passed.")

```